

Project 2 Proposal: Deadlock Detection & Visualization Tool

Group Number: 4

Team Members: Ryan Arnouk, Sunny Nie, Mike Wang, Yingkai Zhao, Alan Zhang

A. Brief Description of Proposed Program Analysis Tool

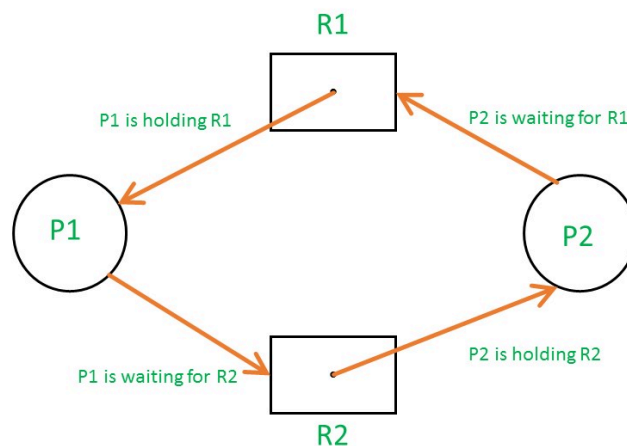
- Our tool performs static analysis of lock dependencies in multithreaded programs by visualizing them as a directed conflict graph. Each node represents a lock, and edges indicate processes/threads acquiring them in a specific order, with cycles in the graph suggesting potential deadlocks. The tool helps programmers verify deadlock risks and promote safe concurrency practices.

B. Intended User-Group

- Developers working on multithreaded, concurrent systems that involve shared resources and synchronization using mutexes.

C. What language will you target? What features will you aim to support?

- Primary Target: C (using Clang)
- Optional Extension: C++ (time permitting)
- Key Features:
 1. Analyzes thread and mutex operations to track lock dependencies in the code.
 2. Generates a directed graph to visualize lock relationships.
 3. Detects and warns about cycles that may indicate deadlocks.



SINGLE INSTANCE RESOURCE TYPE WITH DEADLOCK

Example Output: Directed graph showing a potential deadlock cycle.

(Image source: [GeeksforGeeks](https://www.geeksforgeeks.org/deadlock-detection/))

D. Which questions about program behavior will users be able to answer with your tool?

- Does lock X depend on lock Y being acquired first?
- Can process P, using locks L1 and L2, cause a deadlock?
- Does this program contain conflicting lock requests that could cause resource contention?

E. Which of the three criteria does your project satisfy?

1. Includes a static analysis component

- ☒ **Yes.** Our tool traverses the AST to detect lock initialization, lock/unlock operations, and process dependencies. It does not execute any code under analysis.

2. Summarizes results over multiple executions in a non-trivial manner

- ☒ **Yes.** Our tool does not summarize results over multiple dynamic executions, but it performs static analysis over all possible executions. See F below.

3. Has a substantial visualisation component, to present resulting data in a useful/appealing way

- ☒ **Yes.** The static analysis component will output structured data (e.g. JSON) representing all lock dependencies as a conflict graph. This data will be processed by a visualization tool to generate a clear conflict graph with labeled mutexes and processes.

F. Which of the four impossible properties does your project not satisfy?

1. Analysis is fully automatic

- ☒ **Yes.** Our tool analyses code and generates results without user intervention.

2. Analysis always terminates

- ☒ **Yes.** Our program always terminates because we are not dynamically analyzing the runtime of the program (and therefore running into cases where deadlock/contention could cause the program to stall).

3. Analysis always says “yes” when the answer should be “yes”

- ☒ **Yes, for the question:** “Does this program contain a potential deadlock?” Our tool detects all potential deadlocks by analyzing cycles in the lock dependency graph, ensuring no false negatives.

4. Analysis always says “no” when the answer should be “no”

- ☒ **No, for the question:** “Is this flagged deadlock guaranteed to happen at runtime?” Our tool performs optimistic static analysis, meaning some detected cycles may not actually lead to deadlocks depending on execution order, resulting in false positives.